

NYCU-ECE DCS-2026

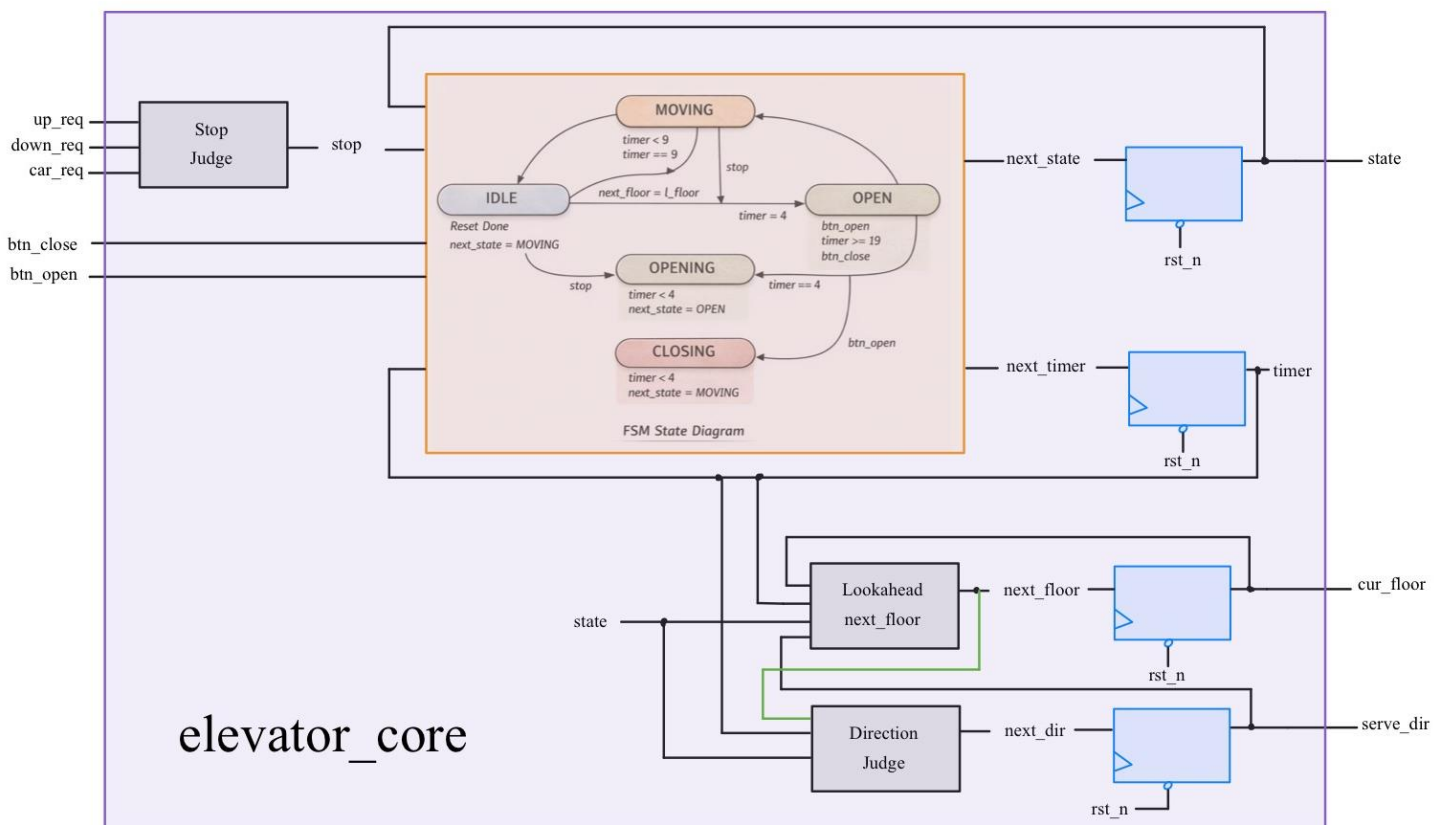
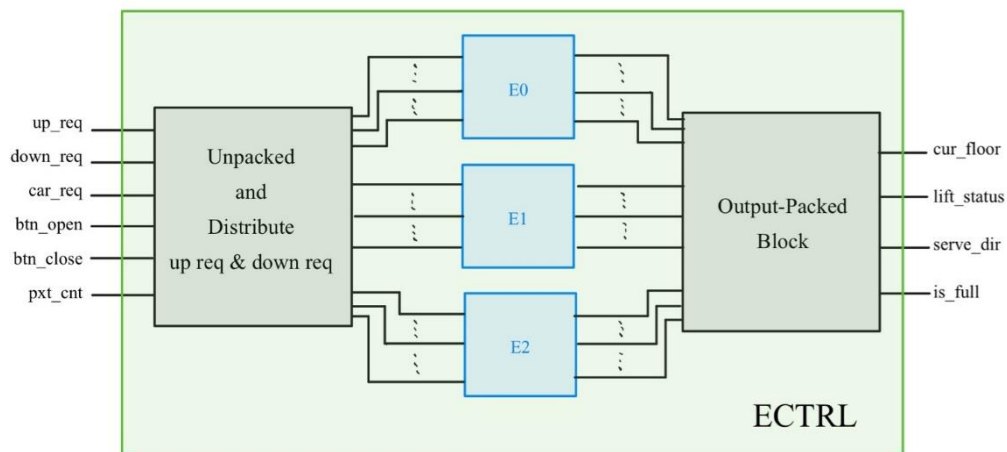
HW03 Report

Name: 陳欣怡

Student ID: 113511116

Server Account: dcs093

Architecture



Design Methodology

透過中央派車系統將外部請求分給各個電梯，再由電梯模組來計算此台電梯的移動狀態、下個樓層、移動方向等個別資訊，最後將三台電梯的輸出組合起來作為 top module 的輸出。

Optimization of Performance :

1. **簡化調度邏輯**：原本是在 top module 透過各台電梯的樓層遮罩去計算外部請求要派給哪台電梯，而後再電梯模組裡又要在執行一次遮罩判斷下一次執行的方向，以及是否還有需求否則進入 IDLE。後面我將電梯內部的調度邏輯改成「永動機」，也就是讓電梯不要進入 IDLE，朝著現有的方向持續行駛，直到碰到頂樓/底樓再轉向，以此大幅簡化電梯模組裡的邏輯判斷，在 latency 從 110W→112W 的情況下，面積能由 1.1W 減少至 9k。
2. **Latency 與 Area 的取捨**：原先在 top module 是對前兩台電梯執行遮罩判斷，剩下沒人接的請求再丟給第三台。後來我發現只需要給第一台遮罩，剩下的給第二台與第三台電梯就好。如此 latency 僅從 1120904 → 1127598，而面積卻小了近 1k！
3. **簡化判斷式**：因為 CLOSING 與 OPENING 最少可以 5cycle 就好，因此將 $\text{timer} < 4$ 判斷改為 “!timer[2]”，可以減少使用比較器，面積小了約 300。

Discussion

原先發現 timer 沒有初始化的必要，因此將它從 rst_n 移出並建一個沒有 reset 的 DFF 給它以減小面積，然而 01、02 都過了 03_GATE 卻報錯，與同學討論後發現是沒有正確初始化的關係，將初始化的部分補齊後就有順利通過 03_GATE。

Reflection

這次作業最困難的地方在於實現方式非常多樣，如果每一種都親自嘗試，會花費相當多時間。因此一開始我先設計了幾個不同策略，交由 AI 產生對應的實作，再透過比較各方案的優缺點來決定方向。在完全依賴 AI 產生的版本中，最佳 performance 約為 $112W \times (1.2W^2)$ ；之後我進一步檢視程式碼並調整策略，才讓效能有了明顯突破。這個過程讓我深刻體會到，AI 工具確實能大幅提升效率，但若缺乏自己的理解與參與，很難達到最佳結果，也讓我更加體認到主動思考的重要性。

AI-use Log

If AI was used, please include the following details:

- <https://gemini.google.com/share/7c7069783590> Gemini
- Key Prompts: 3–5 primary prompts (paste the **screenshots**).

可以幫我改成在最上層判斷由哪個電梯去完成需求嗎，先判斷需求樓層是否跟電梯運行同方向，若是則先把需求樓層判給會到達該樓層的電梯，若都會到達則給距離最近的；若需求樓層與電梯不同向則先等電梯完成單一方向的載程後回到 IDLE，再判給閒置狀態且距離最近的電梯

如果用權重分配呢，同向cost設為0，同向且會抵達該樓層設為-1，滿載則cost設大一點

我應該怎麼做才能讓每個乘客的等待時間最少